Redrawn: how the code works

A guide to the California congressional map site for a reader with strong technical experience but no familiarity with this codebase: the overall architecture, the data, how scrolling drives the map, and what the map shows at each step of the story.

Written October 25, 2026. Describes the code as of that date, before the landing-page slideshow was added. File and line references point to `main.js`, `scenes.js`, `index.html` and `style.css`.

1. Summary
2. Repository layout
3. Data: the closest thing to a backend
4. How the page is layered
5. main.js, section by section
6. scenes.js, index.html, style.css
7. Startup sequence
8. The scrolling system
9. What each scene setting changes
10. The camera
11. The 8 scenes, one by one
12. Sharp edges and open decisions

---

1. Summary

There is no backend. The site is static: one HTML page, one stylesheet, two plain JavaScript files and pre-built JSON data. There is no build step, framework, server code or database. Any static file server can host it (GitHub Pages, Netlify, `python3 -m http.server`). It does need a real server: opening `index.html` directly as a file fails, because the page loads its data with `fetch()`.

The only outside dependencies load from CDNs: MapLibre GL 4.7.1 (the map engine) and two Google Fonts (Libre Caslon Text and Public Sans).

The experience is a scroll-driven story. A map of California's 52 congressional districts is fixed behind the page, and 8 story cards scroll over it. As each card reaches the middle of the screen, the map changes to match it: which map year is shown, whether districts are colored by 2024 results, which districts are outlined, whether the other map's lines show through, and where the camera points.

2. Repository layout

```
index.html        page structure: landing, 8 story steps, take-action section, map overlays
style.css         all visual styling: color tokens, layering, phone layout
main.js           everything that runs: map setup, data join, layers, scenes, tooltip, scroll handling
scenes.js         the story script: one plain object per step saying what the map should show
data/
  districts_2021.json   52 district shapes, 2021 commission map (GeoJSON)
  districts_2025.json   52 district shapes, 2025 Prop 50 map (GeoJSON)
  results_2024.json     2024 presidential votes counted under each map   (used by the app)
  results_2024.csv      the same table as CSV                            (not used)
  temp.txt              empty
styles.css, CONTRIBUTIONS.md   empty leftovers (note styles.css vs. style.css)
```

> **scenes.js was not tracked by git** at the time of writing, and the other three main files had uncommitted changes. Committing everything except `scenes.js` would break the site at load with `SCENES is not defined`.

3. Data: the closest thing to a backend

All data preparation happened before the site runs. The browser only loads finished files and joins them.

District shapes

- Each file is a GeoJSON collection of 52 districts in plain longitude/latitude.
- The two files come from different sources, so their field names differ. 2021 uses `POPULATION`, `CVAP_19`, `F_HSP_CVAP` and so on; 2025 uses `A_POP`, `CVAP`, `HSP_CVAP_P` and so on.
- 2021 districts are single `Polygon`s; 2025 districts are `MultiPolygon`s.
- `DISTRICT` is a string, `"01"` in one file and `"1"` in the other.

Results (results_2024.json)

- Shape: `{ meta, districts: [{ district, "2021": {harris, trump, total, margin}, "2025": {...}, same_area_pct }] }`.
- `margin` = (Harris − Trump) ÷ all presidential votes × 100. Positive means Harris led.
- Source: The Downballot's district-level calculations. The `meta` block records that statewide totals match certified results exactly for 2021 and within 2 votes for 2025.
- `same_area_pct` is the overlap between the old and new versions of the same district number (shared area ÷ combined area), from 0 (a completely different place) to 100 (unchanged). The app doesn't use it yet, but it backs up two story cards: District 41 is **0.0** and District 15 is **99.9**.

The join, in the browser

When the map loads, `main.js` fetches all three files in parallel, then:

1. Indexes the results by district number.
2. Runs `normalize()` on every district shape. It maps either file's field names onto one shared set (`population`, `cvap`, `hispanic`, `asian`, `black`, `white`), converts `DISTRICT` to a number, and attaches `margin` for the matching map year.
3. Keeps the cleaned collections in `planData` so camera framing can be calculated later.

After this step, nothing else in the code knows the two source files were different.

---

4. How the page is layered

The map is a single fixed, full-screen layer behind everything. The rest of the page scrolls over it. The stack, bottom to top:

```
z 0  #map              fixed, full screen, black; MapLibre draws here
z 1  #story            the 8 .step sections; clicks pass through to the map except on cards
z 2  #plan-buttons, #legend, #story-progress   fixed controls
z 3  #tooltip          fixed, follows the mouse
z 4  #landing, #take-action   solid black and positioned, so they fully cover everything below
```

The map is always running; the landing and take-action sections simply cover it. That's why the first scene can be applied instantly at load: by the time the landing scrolls away, the map is already in place.

Mouse wheel, drag and touch zoom are all turned off on the map, so scrolling always moves the page. Users can't pan or zoom; only the story moves the camera. Hover still works because `#story` ignores pointer events while `.card` accepts them.

---

5. main.js, section by section

Constants

- `MARGIN_COLOR` is a MapLibre color expression: `["interpolate", ["linear"], ["get","margin"], -30, orange, 0, #2A2A2A, 30, purple]`. MapLibre evaluates it per district on the graphics card. Values past ±30 get the end color.
- `PLAIN_COLOR`, `PLAIN_OUTLINE` and `RESULTS_OUTLINE` cover the results-off look.
- `PHONE_QUERY` (≤699px) and `REDUCED_MOTION_QUERY` are media queries checked while running.

Map setup

The map uses a hand-written style with no base map tiles, just a black background layer, so there are no tile requests and no API key.

Layers

Built once in `map.on("load")` (line 304) and never removed. Each map year has four layers drawn from source `plan-{year}`: `fill-`, `outline-`, `ghost-` and `highlight-`. They are added grouped by type (all fills, then all outlines, then all ghosts, then all highlights), not grouped by year. That ordering is deliberate: it puts the dashed ghost of one map on top of the other map's fill. Each source sets `promoteId: "district"`, which nothing relies on yet.

The state model

Four small functions each change one aspect of the map, only by toggling visibility, paint properties or filters:

Function	Effect
`setPlan(year)` (97)	Shows one year's layers and hides the other's; updates the active button; re-applies the ghost (always the *other* map); hides the tooltip.
`setGhost(on)` (115)	Shows or hides the faint dashed outline of the map not currently shown.
`setResults(on)` (125)	Switches fills between margin colors and flat gray, switches outline color, shows or hides the legend.
`highlight([n…])` (134)	Sets the filter `["in", ["get","district"], ["literal", nums]]` on both highlight layers.

Module-level `currentPlan`, `ghostOn` and `currentScene` let these be called in any order.

Scenes

`applyScene(scene, animate)` (line 188) is the only entry point the story uses. It calls the four functions plus `moveCamera()`, and writes the scene's `alt` text into a hidden live region (`#scene-alt`) so screen readers announce each map change. It does nothing if handed the same scene object twice in a row.

Tooltip

- Mouse events are attached per fill layer; hidden layers receive no events, so only the visible map responds.
- Two lines are built with DOM methods (`replaceChildren`), never `innerHTML`, so there's no injection risk.
- Margins are rounded to whole points: "leaned Harris by 12 points", or "was about even".

- It's mouse-only, so touch and keyboard users can't reach this information: the project's biggest accessibility gap.

Runs immediately, before the map loads

`numberCards()` adds "N of 8" to each card. `updateStoryProgress()` drives the left-edge progress bar from a passive scroll listener.

---

6. scenes.js, index.html, style.css

scenes.js

The story's script, with no logic: a global `SCENES` array whose objects have `id`, `plan`, `showResults`, `highlight`, `ghost`, `camera` and `alt`. `plan` is a string and is converted to a number in `applyScene`. It loads as a classic script before `main.js` and shares the global scope; there are no modules.

index.html

1. Fixed overlays: map container, plan buttons, tooltip, legend, progress bar, screen-reader live region.
2. The landing `<header>`, including an inline SVG of the real 2025 District 1 boundary (232 points, `pathLength="1"`) that draws itself once on load.
3. `<main id="story">` with the 8 steps.
4. The take-action section: two lists of links that open in a new tab, with hidden text explaining each link to screen readers.
5. The two scripts, at the end of `<body>`, so the page structure exists when they run.

style.css

- Color tokens on `:root` (`--ink`, `--paper`, `--muted`, `--rule`, `--harris`, `--trump`, `--even`) plus `--serif` and `--sans`.
- The map colors are **duplicated** as hex values in `main.js`, because MapLibre can't read CSS variables. Change both together.
- Phone layout (≤699px): steps are 120vh, cards are `position: sticky` at 55% of the screen with a 45% height cap. This must agree with the 45% bottom padding in `cameraPadding()`.
- Reduced motion is handled in three places: smooth scrolling, the outline animation, and the camera (in JS).

7. Startup sequence

```
HTML parsed -> CSS + fonts requested -> maplibre-gl.js loads (blocks the page; it's in <head>)
-> scenes.js defines SCENES -> main.js runs:
     map created (empty black style)
     numberCards(), progress bar listener
     map "load" -> fetch 3 JSON files in parallel -> normalize/join -> add sources + 8 layers
                -> hook up tooltip + plan buttons -> setUpStory():
                     applyScene(SCENES[0], no animation) -> start the IntersectionObserver
```

8. The scrolling system

Scrolling never moves the map directly. The page scrolls normally, and one thing listens: an `IntersectionObserver`. When a story step reaches a line across the middle of the screen, the code looks up that step's scene and sets the map to match. Because the map is fixed behind everything, it looks like the background reacts to your scroll. There are no scroll-position calculations or scroll libraries; only the progress bar runs on every scroll event.

The pieces

```
scenes.js                 index.html                          main.js
SCENES = [                <section class="step"               IntersectionObserver
  { id: "map-2021",  <----  data-scene="map-2021">    --->   (watches every .step)
    plan, showResults,      <div class="card">...                    |
    highlight, ghost,     </section>                                 v
    camera, alt },                                           applyScene(scene)
  ...                                                        |- setPlan()     which map is shown
]                                                            |- setResults()  colors + legend
                                                             |- highlight()   white outlines
                                                             |- setGhost()    dashed old lines
                                                             |- moveCamera()  fly or jump
                                                             '- #scene-alt    screen-reader text
```

A step is linked to its scene only by matching `data-scene` with `id`. The HTML holds the words, `scenes.js` holds what the map should look like, and `main.js` holds how to get there.

The trigger band

`setUpStory()` (line 274) creates the observer with `rootMargin: "-45% 0px -45% 0px"`. A negative root margin shrinks the watched area: here, to a band covering the middle 10% of the screen (45% to 55% of its height).

```
+--------------------------+   0vh
|                          |
+--------------------------+  45vh -+
| //// trigger band /////  |        |- a step is "intersecting" only while it overlaps this band
+--------------------------+  55vh -+
|                          |
+--------------------------+ 100vh
```

The callback ignores steps that are leaving the band and acts only when a step **enters**. No leaving logic is needed: steps are stacked edge to edge, so when one enters, the previous one has just left. Scrolling down, the trigger fires when the next step's top edge rises past 55%. Scrolling up, it fires when the previous step's bottom edge drops past 45%.

When the map changes on desktop

Each step is 100vh tall with its card centered vertically. On a 741px-tall screen with a card about 250px tall, step N+1 triggers when its top reaches 55vh (about 408px). Its card's top is then at roughly 408 + 370 − 125 ≈ **650px**, just appearing at the bottom of the screen. So the map starts changing as the next card **peeks in from below**, and the 1.2-second camera flight roughly overlaps the scroll that brings the card to the middle.

When the map changes on phones

Each step is 120vh tall. The card sits at the top of its step and sticks at `top: calc(55vh + 8px)`, capped at `45vh − 16px` tall. As you scroll:

1. Card N is stuck in the bottom part of the screen; the map is framed in the top 55%.
2. Step N's bottom edge rises and pushes card N up and off screen (sticky elements can't leave their parent).
3. Card N+1 rides up at the top of its own step, right behind it.
4. Step N+1's top, which is also card N+1's top, crosses 55vh. **The scene changes here**, at almost the same moment card N+1 settles into its sticky position.

On phones the map changes as each new card **locks into place**. Three numbers must agree and are not linked in code: the 55% trigger line, the 55% sticky position, and the 45% bottom padding in `cameraPadding()`. The extra 20vh per step gives scroll distance where a card stays stuck and readable while the map is still.

Why every scene describes the whole map

Each scene lists every setting rather than "what changed from the previous scene", and `applyScene()` applies all of them every time. That makes the system reliable:

- **Scrolling backwards** needs no extra code: scene 4 to scene 3 is just "apply scene 3".
- **Anchor jumps work.** "Skip to the map" jumps straight to step 8. Only step 8 enters the band, steps 1–7 never trigger, and scene 8 still produces the right map.
- **Fast scrolling** past several steps may trigger several scenes in a row; the last one fully determines the result.

The same-scene check in `applyScene` matters because the observer can fire more than once for one step (a small scroll back and forth at the edge). Without it, every repeat would restart the camera flight.

The progress bar

```
through = (screen height / 2 − story.top) / story.height      -> clamped to 0..1
#story-progress.style.transform = scaleY(through)
```

It's 0 while the top of `#story` is below the middle of the screen and 1 once the bottom is above it. A `scaleY` transform avoids layout work on each scroll event. It sits at z-index 2, hidden under the landing and take-action sections. It measures from the 50% line while scenes trigger at 55%, so it shows overall progress, not the current step; the step number is the "N of 8" line on each card.

---

9. What each scene setting changes

None of these rebuild anything. All 8 layers exist from load; a scene change only flips visibility, swaps a paint value or swaps a filter.

Setting	Function	What happens in MapLibre	What you see
`plan`	`setPlan`	Sets `visibility` on fill, outline and highlight layers for each year	The other year's districts replace the current ones in one frame. Active button updates; tooltip hides.
`showResults`	`setResults`	`fill-color` becomes `MARGIN_COLOR` or flat `#141414`; outline becomes black or `#4A4A4A`; toggles `#legend.hidden`	Districts turn purple, gray or orange by margin, or return to flat gray. The legend appears only when colors mean something.
`highlight`	`highlight`	Sets the district-number filter on both highlight layers	2.5px warm-white outlines on just those districts.
`ghost`	`setGhost`	Shows `ghost-{other year}`, hides the rest	The other map's lines appear dashed over the current one.
`camera`	`moveCamera`	`cameraForBounds`, then `flyTo` (1.2s) or `jumpTo`	Zooms to the whole state or to one or more districts.
`alt`	(inline)	Writes into `#scene-alt` (`aria-live="polite"`)	Nothing visible; screen readers announce the new map.

Order of calls inside applyScene

`setPlan` runs first and internally calls `setGhost(ghostOn)` with the *previous* scene's value, so the ghost switches to the correct "other" year right away. A moment later `applyScene` calls `setGhost(scene.ghost)` with the new value. Everything runs in one JavaScript task, so the in-between state is never drawn.

What animates and what doesn't

- **Animated:** only the camera. Fill colors, visibility and filter changes take effect on the next frame.
- **Possible exception:** MapLibre has a default ~300ms fade for paint changes between two fixed values, so the outline color switch may fade slightly. The switch between flat gray and data-based fill colors is effectively instant.

- **Reduced motion:** checked on every camera move; `jumpTo` replaces `flyTo`. Smooth scrolling and the landing animation are turned off in CSS.

---

10. The camera

```
scene.camera
 |- "state"        -> CALIFORNIA_BOUNDS (a fixed bounding box)
 '- { fit: [1] }   -> districtBounds(currentPlan, [1])
                        for each matching district in planData[currentPlan]:
                          flatten coordinates (.flat(1) for Polygon, .flat(2) for MultiPolygon)
                          take min/max of lng and lat -> [[west,south],[east,north]]
        |
        v
 map.cameraForBounds(bounds, { padding: cameraPadding() })
        |                          |- desktop: left = card's right edge + 40px
        |                          '- phone:   bottom = 45% of screen height
        v
 flyTo(target, 1200ms)  or  jumpTo(target)
```

- **Districts are measured on the map being shown.** `setPlan` runs first, so `{fit:[41]}` with plan 2025 frames the *new* District 41.
- **Padding keeps the subject clear of the cards.** On desktop the card's position is measured live with `getBoundingClientRect()`; on phones it's a fixed 45%.
- **Interrupted flights** stop where they are and the new flight starts from there, so fast scrolling looks like one continuous move.
- **The first scene doesn't animate.** It's applied instantly behind the landing so the map is ready when the first card arrives. `cameraForBounds` can return nothing when padding exceeds the screen; that case is guarded.

11. The 8 scenes, one by one

#	Scene	Map	Colors	Outlined	Old lines	Camera
1	districts	2021	off	none	off	whole state
2	why-redraw	2021	off	none	off	whole state
3	map-2021	2021	**on**	**1, 3, 9, 13, 41, 48**	off	whole state
4	map-2025	**2025**	on	1, 3, 9, 13, 41, 48	**on**	whole state
5	district-1	2025	on	**1**	on	**zoom to 1**
6	district-41	2025	on	**41**	on	**zoom to 41**
7	district-15	2025	on	**15**	on	**zoom to 15**
8	explore	2025	on	**none**	**off**	**whole state**

Bold marks what changed from the scene above. Each step changes one or two things, giving the reader one new idea per card.

Page load: scene 1, hidden

Once data loads, scene 1 is applied with no animation while the landing still covers the map: all 52 districts of the 2021 map in flat #141414 with #4A4A4A borders, framed to the whole state. The legend stays hidden.

1 → 2: nothing visible changes

The two scenes are identical. The same-scene check compares objects, not contents, so scene 2 is applied, but every setting already matches and the camera "flies" to where it already is. The map stays still while the reader reads about Texas and Prop 50.

2 → 3: colors appear and six districts are outlined

The fill switches to the 2024 results scale and borders go black. White outlines appear on districts 1, 3, 9, 13, 41 and 48. The legend appears and stays through scene 8. Under the 2021 lines all six lean Trump, so they show in orange tones. The camera doesn't move.

3 → 4: the map switches (the key moment)

- The 2021 layers disappear and the 2025 layers appear in the same frame.
- The dashed 2021 lines appear over the 2025 map.

- The outline list is unchanged but matched by number, so the outlines **jump to the new shapes** of the same six districts.
- The camera stays on the whole state, showing the full redraw at once.

These six are exactly the districts that flip, all from Trump to Harris, none the other way, taking Harris-leaning districts from 41 to 47:

District	2021 lines	2025 lines	How it looks after the switch
1	Trump +24.9	Harris +12.2	orange to medium purple
3	Trump +3.8	Harris +10.2	faint orange to medium purple
9	Trump +1.8	Harris +11.4	near-gray to medium purple
41	Trump +5.9	Harris +14.2	faint orange to medium purple
48	Trump +15.3	Harris +3.4	orange to **only slightly purple**
13	Trump +5.4	Harris +0.4	faint orange to **essentially gray**

> The scale runs from gray at 0 to full purple at +30. District 13's +0.4 is about 1% of the way to purple and looks the same as "even"; District 48 is barely tinted. Two of the six won't obviously look flipped; the white outline is what tells the reader to look there.

4 → 5: first close-up, District 1

The outline narrows to District 1 and the camera flies 1.2s from the whole state to the 2025 District 1 in Northern California. The dashed 2021 lines stay on for direct comparison. Old and new overlap by only **30.9%**, and the new district sits about half a degree farther south, matching the card's "same amount of people, but … not the same community".

5 → 6: the long flight to District 41

- MapLibre's `flyTo` zooms out partway, travels south, then zooms in to the 2025 District 41, centered roughly in southeastern Los Angeles County.
- The new District 41 is small, about 0.3° across. The old one was about 1.4° wide and centered over Riverside County, roughly 100 km east.
- **The camera frames the new district, so the old District 41 is off screen or at the edge.** The dashed lines show old boundaries around the new district, not the old District 41 itself.
- The data supports the card: shared area between old and new District 41 is **0.0%**.

6 → 7: home, District 15

The camera flies to San Mateo County. Old and new overlap **99.9%** and the margin is Harris +47.9 under both maps, so the dashed 2021 lines sit almost exactly under the 2025 outline. The lack of a

second shape makes the "nothing changed locally" point visually.

7 → 8: zoom back out to explore

The camera returns to the whole state, the outline clears and the dashed lines turn off, leaving the plain 2025 map with results colors for free hovering. The take-action section follows and covers the map.

Scrolling back up

8 → 7 flies back into District 15 and restores the dashed lines and outline. 4 → 3 switches back to the 2021 map, turns off the dashed lines, and snaps the six outlines back to their old shapes. 3 → 2 removes the colors, outlines and legend. Scrolling all the way to the landing leaves the map on scene 1's state, ready when the landing uncovers it.

Throughout all scenes

- The two map buttons show which map the scene uses (2021 for scenes 1–3, 2025 for 4–8).
- Screen readers hear each scene's `alt` sentence on change.
- The tooltip works in every scene, for whichever map is displayed.
- The progress bar fills steadily from scene 1 to 8, by scroll position.

---

12. Sharp edges and open decisions

Sharp edges

- **Window resize doesn't reframe the camera.** `moveCamera` only runs on a scene change, and the same-scene check blocks re-running it, so rotating a phone or crossing 699px keeps the old framing until the next step.
- **Scrolling before the data loads.** The observer starts only after all three JSON files (about 1.5 MB) load. Until then the map stays black; afterwards the observer reports the current step and the map catches up.
- **No load error handling.** If a fetch fails, the map stays black with no message.
- **Stale tooltip during a flight.** The tooltip updates only on mouse movement, so it can show the old district while the camera moves under a still mouse.
- **Plan buttons are overridden** by the next scene change; a scene re-applied at a step boundary can undo a visitor's choice.
- **Phone card overflow.** At 375px wide, 6 of the 8 cards are slightly taller than their space and scroll inside the card.
- **No tests, linting or build.** The safety checks are `node --check main.js` and the console warning for a step with no matching scene.

- **Untested in automation.** In an automated Chrome session, scroll-triggered scene changes didn't fire while calling `applyScene` by hand worked, likely because the automated tab wasn't rendering. Confirm once in a normal browser.
- **Cleanup candidates:** empty `styles.css`, `temp.txt` and `CONTRIBUTIONS.md`; a committed `.DS_Store`; the CSV that duplicates the JSON; a stale "light gray" comment in `scenes.js`.

Open decisions

1. **Scene 2 has no visual change.** Make the map respond, or accept it as a pause.
2. **Districts 13 and 48 barely change color** though the text says they flip. Options: lower the scale's maximum from ±30, make "even" lighter, or note in the card that two flips are narrow.
3. **The District 41 close-up doesn't show the old District 41,** though the 0% overlap is the card's point.

Adding a step

1. Add `{ id, plan, showResults, highlight, ghost, camera, alt }` to `SCENES`, filling in every field.
2. Add `<section class="step" data-scene="my-step"><div class="card"><h2>…</h2><p>…</p></div></section>` to `#story` in position.

Numbering, progress, trigger and camera framing adjust automatically. A mismatched id logs `No scene in scenes.js with id "…"` and that step does nothing.